

MultiSportPredict — Project State

Written: 2026-09-08 **Supersedes:** HANDOFF.md (2026-08-29). That file is stale and contains claims that are no longer true (e.g. "MLB grades automatically; other sports need the CSV route" — tennis and NCAAF are now automatic too; the 14-graded baseball record it cites is long since obsolete). HANDOFF.md now just points here — do not maintain both.

Read this before touching the codebase. It is written to be skimmed once and referenced repeatedly, not read start to end.

1. What changed, by area (commit hash — what was broken — what it means)

All ten commits below landed in this one session, on top of a repo where **nothing from this entire multi-day working conversation had been committed before now** — everything was still sitting as uncommitted working-tree changes until today. That gap is closed. It should not reopen: commit as you go from here, not at the end of a session.

Commit	Area	Was broken	Means for output
6880b02	Soccer resolver + soccer/tennis auto-odds	Team-name resolver had no concept of league/tier — "FC Utrecht Youth" silently resolved to senior "Utrecht", wrong club. Tennis had no odds ingestor at all; soccer only fetched odds with an explicit flag. A run with no odds produced "PASS, model probability only" and got manually re-run once a price was typed in — which is why the same game could get pushed twice with contradictory verdicts.	Cross-league name collisions now refuse instead of guessing. Soccer/tennis predictions are now built against a real market price by default, not a placeholder, unless neither is available (state made explicit, never silently blank).
2065fb6	NCAAF results ingestor	No NCAAF results feed existed; <code>AUTO_SOURCES["ncaaf"]</code> aliased the NFL endpoint (wrong sport).	FBS results (rank-1-25 only, FCS/D2/D3 excluded) are ingestible and backfilled for the season-opening weekend. Results only — no predictor exists.

Commit	Area	Was broken	Means for output
78d1f72	Grading: NCAAF + tennis wired; <code>tier_of()</code> fixed	Tennis had zero automatic grading, ever (25 rows logged since Aug 23, 0 ever graded). Separately, <code>tier_of()</code> only recognized soccer's exact "BET" string — every tennis/MLB moneyline recommendation, real bets included, was silently counted as "informational" in every win-rate report that used this helper.	Tennis and NCAAF results now grade automatically. Any win-rate report run before this commit undercounted real bets — see §3.
070e940	3 soccer leagues + ESPN host fix	Saudi Pro League, Turkish Süper Lig, Eerste Divisie had no data at all. Separately, <code>ingest_soccer_espn.py</code> was calling an Akamai-blocked host for the <code>requests</code> library — likely silently breaking Liga MX/Brasileirao/Argentina too, since those have no fallback source.	3 new leagues have real team data (18/18/20 teams). The host fix may have also quietly restored leagues that were failing before — not separately verified.
a631bda	MLB moneyline wiring + sign bug fix	Moneyline odds were fetched and stored, then never compared against the model — no real edge, ever, for this market. Separately: a live pushed card read "St. Louis Cardinals 45.2% ... LEAN St. Louis Cardinals ML (edge: -9.1%)" — the pick was correct, but the displayed edge was the <i>other</i> team's number, unsigned to the actual recommendation.	MLB moneyline now shows a real, priced edge. Any moneyline recommendation is now signed to the side actually picked — no more "recommends X, shows Y's negative edge." This is a brand-new feature as of today; it could not have corrupted anything from before today (confirmed: zero MLB moneyline rows exist in the DB before this session).
61da005	NFL: same sign bug	Identical pattern to MLB, independently present in both NFL's spread and moneyline blocks.	Same fix, same guarantee.

Commit	Area	Was broken	Means for output
350416b	Soccer: remove fake shots/tempo/pressure constants	shots, sot, tempo, width_crossing, final_third_pressure were fixed home/away constants for every soccer match this project has ever predicted , verified by comparing unrelated matchups and finding them byte-identical. They fed moneyline, totals, and BTTS. Corners' total (corner_projection) had <i>no other input</i> — confirmed the literal constant 10.9 for every match tested.	Moneyline/totals/BTTS now run on real signal only (xG-derived) — see §4 for the measured size of the shift, which is real and worth watching, not fully resolved. Corners is retired from output entirely (next row), not repaired — there was nothing real left once the fake terms were removed.
0d874dc	Canonical embed formatter; tennis phantom-edge fix; supersede dedup	Eight independent, drifted Discord embed builders existed; basketball had no extractor at all and was silently [REFUSED]. Tennis's displayed "edge" was computed against a hardcoded 50% baseline even with zero market odds supplied , indistinguishable from a real edge. Discord dedup was content-hash only — it could never catch "same game, different inputs," which is <i>why</i> the same game could get pushed twice with different verdicts.	One shared formatter; basketball reaches Discord now. Tennis with no real odds now explicitly says so instead of showing a fabricated-looking edge. A second push for a game already pushed today edits the existing card instead of adding a contradictory second one.
5f812d9	Hitter props (MLB), real data	Team-level "Home runs (H/A)" was the only per-batter-adjacent market, and the configured player-stats path (ingest_mlb_props.py → pybaseball → FanGraphs) is dead (403, same Cloudflare block that killed FBref).	Real per-player hit/HR projections from statsapi.mlb.com, verified against a real player. Not yet wired into the embed — see §5.

Commit	Area	Was broken	Means for output
6ad28b3	Small MLB completion	<code>run_mlb.py</code> fetched odds and printed them but never passed them into the function that needed them.	Closes a gap in the moneyline-wiring commit above.
5262e79	Preserved <code>import_picks.py</code>	Not this session's work — see §6.	A complete, working tool for tracking a third party's picks without contaminating Ron's own record. Untouched, uncommitted before now.

2. Current state, per sport

Sport	Data	Markets genuinely priced	Odds auto-fetched	Grading wired	Graded record
MLB	Real (statsapi.mlb.com)	Moneyline (new today, real edge), totals, NRFI. Hitter props built, not wired to output. Pitcher FIP/BB9 are dead constants (§4).	Yes, already worked before today	Yes, automatic	46 Aug rows retroactively regradable, but none were ever issued as a real "BET this" recommendation — see §3
Soccer	Real, xG estimated from goals (tier 2) everywhere	1X2, totals, BTTS — real signal, contaminated-then-partially-repaired today (§1). Corners retired, no real input exists.	Yes, as of today, 12 of 13 leagues (Eerste Divisie fully repaired)	No sport-specific gap — soccer grades via the generic total/moneyline/BTTS path	21 of 37 Aug rows graded: 8-13-0, -5.73 units
Tennis	Real (ATP/WTB match history + ESPN odds)	Moneyline only. Set-market derivatives are model-computed, not fed by a real odds feed (no set-betting market exists on The Odds API).	Yes, as of today (tournament-keyed, empty between majors — explicit state, not a silent gap)	Yes, as of today	16 of 25 rows graded: BET 2-0, LEAN 3-0, PASS 3-2, INFO 4-2 (overall 12-4, but see §3 for why that headline number is the wrong one to quote)
NCAAF	Results only, real	None — no predictor exists	N/A	Yes (results ingestor + grading), as of today	No predictions exist to grade yet
Basketball/EuroLeague	Real	Spread, total	No	Not verified this session	Not verified this session
NFL	Real (prior season deliberately, per its own design)	Spread, moneyline, total	No	Not verified this session	No NFL rows exist in the DB at all

3. The honest record — read this before quoting a number to anyone

Tennis: 12-4 overall (75%), +6.91u, +43.2% ROI — but only 5 of those 16 graded rows were ever a real recommendation. Split by tier: **BET 2-0, LEAN 3-0**, PASS 3-2 (informational), INFO 4-2 (no market was ever supplied for these). The 12-4 headline blends real picks with rows that were never presented as a bet. 5 decisions on a 25-row sample is not evidence of anything yet, in either direction.

Soccer: 8-13-0, -5.73 units, from 21 of 37 August rows graded. A real, negative, small-sample record. Only 5 of the 37 rows were ever flagged "BET" at push time; the rest are graded PASS rows included for calibration, not picks.

Baseball's 46 August "total" rows are not what they look like. Every one was stored with the literal recommendation text "Over : 63.9% | Under : 36.1%" — a raw readout, not a decision — because they predate the fix that made this field say "OVER"/"UNDER"/"PASS". Grading still produces a 17-12-1, +3.45u record because it derives the pick from the stored numeric `model_value`/`market_value` at grade time, not from that text — so the win/loss math is real. **But none of these 46 predictions were ever actually presented to Ron as an actionable "bet this" at the time they were made.** The record is a retroactive reconstruction of what the current logic would have said, not a log of picks that were given.

No CLV tracking exists anywhere. Checked the schema directly — there is no `clv` column, and nothing computes it. If commercial positioning implies CLV is measured, it isn't.

Bottom line for anyone considering this a track record to sell against: none of the three sports currently has a record that is both real and large enough to mean anything. Tennis and soccer are real but tiny. Baseball's number is numerically real but was never actually issued as picks. State all three caveats together, every time, or don't quote the numbers.

4. Open issues, ranked

1. **mcp_server.py / msp_bot.py are incompatible, CONFIRMED as the live cause of stale Discord output, not just a restart risk.** msp_bot.py calls `tools.TOOLS[name]["handler"]` — a module-level dict. The mcp_server.py now on disk has no such dict, only a class (MCPServer) with one tool (`tool_predict_match`, soccer-only), built for Cline/Cowork integration. Confirmed 2026-09-09: msp_bot.py (PID 18840) has been running continuously **since 8/29/2026 11:11 PM** — 11 days, checked via `Get-CimInstance Win32_Process`. Python doesn't hot-reload; every `!tennis!/mlb!/push` command since Aug 29 has been served by 11-day-stale code, meaning **none of the fixes made in any session since then — including the WTA best-of-5 fix below — have ever reached Discord through the bot.** This is very likely the reason a bug that looks fixed in the code can still show up live. Restore plan, scoped 2026-09-09, not yet executed (Ron's call): `git log --all --follow -- mcp_server.py` shows only two commits ever touched this file — 421caee (pre-session baseline, has a real 15-tool `TOOLS` dict matching every one of msp_bot.py's 11 required tool names exactly: `list_teams`, `team_stats`, `list_players`, `run_tennis`, `run_mlb`, `run_slate`, `push_to_discord`, `push_tennis`, `record`, `pending_results`, `check_data`) and 6880b02 (this session's own soccer-resolver commit, which swept in the class-based rewrite as one of the documented accidental extra files — see §5 — not deliberate work by this session). Restored the 421caee version to a scratch file and ran its own `--selftest`, plus called several handlers directly with the exact kwargs msp_bot.py sends (`run_tennis` with a real matchup, `team_stats`, `check_data`): all passed against **today's** scripts and data with zero drift found across every subprocess target (`run_tennis.py`, `run_mlb.py`, `run_soccer_batch.py`, `data_guard.py`, `grade_predictions.py`) and every directly-read store file. mcp_server.py currently has **no uncommitted changes** (checked immediately before touching anything), so restoring it is not a clobber risk right now — but restoring would discard the soccer-only class currently in the last commit, and nobody has confirmed who wrote it or whether Ron or the other active agent is relying on it for a Cline/Cowork workflow. That's Ron's call, not an inferred one. **Ron must not restart the bot under any circumstances until this is resolved and he's told it's safe** — restarting onto today's incompatible file would break every bot command immediately, trading "stale but working" for "fully broken."
2. **Tennis best-of-3 vs best-of-5 was hardcoded downstream of a correct detection, and there are two independent detections of it.** Fixed 2026-09-09: `predict_tennis_match()` correctly branched the *underlying probability* by `best_of_5`, but handed it to `_set_rec()`, which hardcoded the *display text* to "3.5 Sets" regardless of format — reproduced on a real WTA match (Sabalenka vs Swiatek), which came back "LEAN OVER 3.5 Sets," a result that cannot happen in a best-of-3 match. `_set_rec()` now takes `best_of_5` and picks the real line (2.5 vs 3.5). Regression test added: `tests/test_tennis_set_format.py`, 6/6 passing. Also fixed a matching dangerous default in `batch_tennis.py` (`match.get("best_of_5", True)` → now requires the key explicitly). **Not consolidated, still a live overlap risk:** `run_tennis.py` (tour membership: `tours == {"atp"}`) and `predict_match.py` (its own, separate `league_lower` in `gs_keywords` check) each independently decide `best_of_5` for their own CLI entry point. Both are safe today, but two implementations of the same decision will drift again — worth consolidating into one function the next time either is touched.
3. **Corner formula has no real input at all**, not just a contaminated one. Retired from the embed (commit 0d874dc) rather than repaired, because there was nothing left to repair — see commit 350416b's note.
4. **Soccer totals may need a compensating re-fit, not just the constant removal.** The strip dropped projected totals 27-38% across three real matchups. The after-numbers are individually defensible, but this was a mechanical strip, not a recalibration, and hasn't been checked against real market totals fetched via today's new auto-fetch to see if the model now sits systematically below the market. **Check this before trusting soccer totals for anything sized.**
5. **Odds API quota is a real, if now-manageable, constraint.** Free-tier assumption (~500/month, inferred from usage headers, not confirmed on Ron's actual dashboard). Measured real cost: 1 credit/market/call, cost

independent of how many games are in a league that day. With today's caching (once per league per calendar day) and skip-if-no-fixture design, realistic usage is roughly 6-10 credits on an active day — comfortably under budget, but Ron is separately raising the quota question with the provider; build against whichever plan he lands on.

6. **Eerste Divisie is modellable but unpriceable.** Real team data exists (20 teams), but The Odds API has no Dutch second-tier key under any name — checked the full sports list directly. Predictions for this league will never carry a real market edge until a different odds source is added.

7. **Champions League is currently unpriceable too, but for a different, fixable reason.** Four real UCL matches were run on 2026-09-08 (AEK Athens–LASK, Dortmund–Villarreal, Porto–Man City, Club Brugge–Aston Villa); all four had to run on manually-typed market totals because `live_odds.py`'s `SOCCER_LEAGUES` had no "champions league" entry at all — an unverified gap, not a confirmed one like Eerste Divisie above. The live discovery check needed to confirm whether The Odds API's `soccer_uefa_champs_league` key exists on this account's plan was blocked before it could run. Added an explicit `covered: False` row with that exact caveat in its reason string, so a UCL fetch now fails loud instead of silently returning nothing — but this is a "go run the check" item, not a "no data exists" one. Once verified, wiring it in is probably one line plus a credit-cost report, the same shape as the Saudi/Turkish additions earlier today.

8. **The CLI's `--live-odds` flag is broken and has been returning nothing for a while.** Separate from everything above: `--live-odds` triggers a *different*, older code path (`_fetch_live_soccer_market`, hits `api.opticodds.com`) than this session's own `live_odds.py` auto-fetch. It 404s on every call — the URL itself is malformed (`/api/v3/v4/sports/...` mixes two API versions in one path). Confirmed on all four UCL matches today; not something built or broken by this session. Whoever runs this pipeline day to day is passing a flag that does nothing and getting a printed `[ERROR]` line instead of odds — worth either fixing the URL or removing the flag so it stops looking like it's supposed to work.

9. **Pitcher FIP and BB9 are dead constants in the MLB moneyline path**, found during a correctness pass but not fixed: `models/baseball_predictor.py` reads them via `kwargs.get(..., <constant>)` and nothing in the call chain ever passes them — same bug shape as soccer's shots/tempo, smaller blast radius (team-level runs/era/whip/obp/slg are real and do flow through correctly).

10. **Frauen-Bundesliga: zero coverage, not thin coverage.** Neither ESPN nor football-data.co.uk has ever heard of it. This is a new-provider decision for Ron, same category as the quota question — don't build toward it without that decision made first.

11. **The overlap map** (full detail was reported separately, summarized here): 8 competing Discord embed builders existed before today (1 now canonical, ~150 one-off scripts still use whichever legacy one they were written against — those weren't touched, they're not what Ron runs); 4 soccer ingestors write to the same file with no marker for which is authoritative (2 are dead — FBref/RealGM both 403 — nothing says so at a glance); `tier_of()` (fixed this session); dead legacy engine files at repo root (`MultiSportModel.py`, `multi_sport_engine.py`, `EuroBallMLModel.py`) that nothing imports but that a future editor could mistake for live. Recommended consolidation order, if picked back up: embed routing for whatever Ron actually runs next → game-identity dedup design conversation → delete or clearly mark the dead files.

12. **One real, unexplained tennis grading gap.** Keys vs Zheng (Sept 5) is confirmed completed on ESPN's own scoreboard but did not match during grading. Not chased down. Everything else in the 9 unmatched rows has a clear, benign explanation (not yet final, duplicate logging, a different name-storage format) — this one doesn't, yet.

5. Concurrent editing — for whoever reads this next

This is real, not speculative, and it is why this document exists in addition to closing the commit gap. Evidence, in order of directness:

- **HANDOFF.md (the document this supersedes) was itself addressed to "whoever picks this up next (Gemini, another assistant, or future me)."** This project has never assumed single-agent ownership.
- **A Google Cloud Code CLI process (`ccloudcode_cli.exe`) was observed running on this machine during this session.** Working directory not confirmed, but its existence combined with the next point is not a coincidence.
- **Two files were found mid-session that this session did not create:** `add_debug_f5.py` (a script that injects a debug print into `predict_match.py`'s "F5"/first-5-innings market handler — the print is now present in the committed code, harmless, left in place) and `import_picks.py` (a complete, unrelated, working feature — preserved and committed in 5262e79, credited as not this session's work). Both timestamped mid-session, both in a style consistent with this project's existing prose conventions, neither something this session wrote.
- **A stray, unterminated debug f-string had made `predict_match.py` entirely un-importable** before this session found and removed it (commit a631bda) — not something this session's own tooling would produce.
- **Git commits in this repo are auto-tagged [AI n%]** by a hook — visible on every commit made today and in this repo's prior history. That tagging convention itself implies multiple, tracked contributors are expected.

Practical consequence: an uncommitted change is not safe here the way it would be in a single-agent repo. Commit as you finish a logical unit of work, not at the end of a long session — that gap is exactly what let ten commits' worth of work sit at risk today. If you observe a live conflict (a file changes under you mid-edit), stop and reconcile before continuing; don't assume you're the only writer.

Left deliberately uncommitted, not part of this session's work, not touched: `BUNDESLIGA_REPORT.md`, `IMPLEMENTATION_COMPLETE.md`, `patch_toulouse_lille_odds.py`, `run_nfl.py`, `run_slate_0906.py`, `slate_paderborn_freiburg.json`, `slate_user_matches.json`, `ingest_nfl_schedule.py` — all predate this session by 2-5 days (file timestamps Sep 3/5/6), unrelated to anything above. `add_debug_f5.py` and a raw log capture (`_full.txt`) from the concurrent activity above are also left as-is — evidence, not deliverables.

6. Recommended next steps, in priority order

1. **Resolve the `mcp_server.py/msp_bot.py` mismatch — scoped and verified 2026-09-09, restore plan ready, needs Ron's sign-off before executing (see §4.1).** Confirmed as the reason 11 days of fixes have never reached Discord. Ron must not restart the bot until this lands.
2. **Check the soccer totals re-fit question (§4.4)** using real market totals now that auto-fetch exists — this determines whether soccer totals are trustworthy or need another pass.
3. **Wire hitter props into the embed and test against a real posted lineup** (this evening's games, not blind). The module and formula are done; the display and end-to-end verification are not.
4. **Decide the two provider questions that are blocking real work:** Frauen-Bundesliga coverage and the Odds API quota tier. Both are Ron's calls, both are currently the reason something can't be built further.
5. **Chase the Keys/Zheng grading gap** — small, but it's the one loose thread in an otherwise fully-explained set of ungraded rows.
6. **Consolidate the two independent best-of-5 detections (§4.2)** and pick up the overlap map's consolidation order (§4.11) once the above is settled — real technical debt but nothing on it is currently producing a wrong

number the way item 1 was.